

Combined Feature Inventory for an Accounting-First BusinessOS Platform

Executive summary

This report inventories the **combined platform** in the order you requested: first the **accounting system** modeled on core TallyPrime-like capabilities, then the broader **BusinessOS AI** feature set layered around it. Team size, budget, primary country, payment rails, deployment regions, and AI model vendors are **unspecified**. That is strategically important because accounting scope changes materially with jurisdiction, tax rules, bank feeds, and filing obligations; likewise, AI vendor choice changes cost, privacy, and model-governance controls.

The recommended platform shape is an **accounting-first modular monolith** with strong tenant isolation and an evented integration layer. The accounting core should be treated as the system of record: invoices, bills, payments, journal entries, bank reconciliation, taxes, ledgers, and reports. BusinessOS AI should sit around that core as the system of engagement: CRM, website/CMS, forms, projects, files, proposals, portal, automations, analytics, and AI assistance. The reason is operational rather than stylistic: accounting needs immutable posting, maker-checker approval, period locks, reconciliation, and auditability; business workflows need speed, flexibility, and cross-object automation. TallyPrime's official help emphasizes voucher-based accounting, bank reconciliation, GST reconciliation, and e-invoicing/returns, while ERPNext, Odoo, Akaunting, and LedgerSMB all center their accounting stacks around chart of accounts, journals, reconciliation, taxes, and financial reporting. [1]

The strongest free/open-source implementation path is: [Next.js App Router](#) + [Tailwind CSS](#) + [shadcn/ui](#) + [Radix Primitives](#) on the frontend; [PostgreSQL RLS](#) with [Neon](#) or [Supabase](#) for relational storage; [Auth.js](#) first and [Keycloak](#) when enterprise SSO becomes necessary; [pg-boss](#) first and [BullMQ](#) only when Redis-backed job throughput is justified; self-hosted object storage via [SeaweedFS](#) or managed object storage via [Cloudflare R2](#); product analytics via [PostHog](#) or [Matomo](#); BI via [Metabase OSS](#); optional AI gateway via [LiteLLM](#); and optional media via [tusd](#), [FFmpeg](#), [Shaka Packager](#), [Video.js](#), and [OvenMediaEngine](#). Security and accessibility should be verified against [OWASP ASVS](#), [OWASP LLMSVS](#),

[NIST AI RMF](#), [WCAG 2.2](#), and [WAI-ARIA APG](#). [2]

Consolidated feature matrix

Feature	Product area	Priority	Complexity	Description, user/data flow, validations, UI surfaces, backend/data, tools, security, ops
Company, tenant, fiscal setup	Accounting	P0	M	User creates company, selects fiscal year, currency, localization, tax registrations, and chart template. Data flows into tenants, companies, fiscal_years, periods, currencies, tax_registrations. Validate uniqueness of company identifiers, open-period boundaries, and localization compatibility. UI: onboarding wizard, company settings, localization selector. Backend: tenant service, configuration service, localization loader. Tools: Next.js , Auth.js , PostgreSQL RLS , Neon . Controls: RLS, audit trail, maker-checker for localization changes, encryption at rest/in transit. Ops: tenant provisioning, backups, region selection is unspecified .
Chart of accounts and dimensions	Accounting	P0	M	User installs or imports COA, sets groups, tax accounts, cost centers, branch/project dimensions. Data lands in accounts, account_groups, dimensions, opening_balances. Validate account code uniqueness, parent-child type consistency, and posting/non-posting account rules. UI: tree table, account drawer, import wizard. Backend: accounting configuration service. Reference patterns come directly from TallyPrime, ERPNext, Akaunting, Odoo, and LedgerSMB docs. Tools: TanStack Table , React Hook Form , Zod , ERPNext docs , Akaunting

Feature	Product area	Priority	Complexity	Description, user/data flow, validations, UI surfaces, backend/data, tools, security, ops
				docs?turn0search3?&url?Odoo docs?turn0search10?&url?LedgerSMB docs?turn11search0?&url?Odoo . Controls: period-safe changes, audit log, approval on account merges/deletes. Ops: import quality and referential cleanup are major migration cost drivers.
Journals, vouchers, posting, reversals	Accounting	P0	H	User drafts a journal or voucher, line items are validated, totals must balance, then a reviewer posts it or it auto-posts based on policy. Data flows from <code>journal_entries/voucher_headers</code> and <code>journal_lines</code> into derived ledger views. Validate debit=credit, period open, dimension validity, tax code compatibility, and immutable-posting rules. UI: journal list, draft form, review/post screen, reversal flow. Backend: posting engine, ledger service, approval workflow, outbox events. Tools: ?url?CodeMirror?turn11search1?&url?pg-boss?turn3search3?&url?pg-boss for advanced expression fields, ?url?pg-boss?turn3search3?&url?pg-boss for background posting/export jobs. Controls: maker-checker, period locks, append-only posting, hash chaining optional, encrypted attachments. Ops: this is the core correctness boundary and should remain in one transactional service. [3]
Sales invoices, credit notes, AR	Accounting	P0	M	User creates invoice, system computes lines, discounts, taxes, due dates, approval checks, sends customer copy, then posts AR and revenue journals. Data uses <code>customers</code> , <code>sales_invoices</code> , <code>sales_invoice_lines</code> , <code>credit_notes</code> , <code>ar_allocations</code> . Validate customer/tax status, duplicate numbering, tax rules, credit-note limits, and period state. UI: invoice list, invoice create/review, customer detail, PDF preview. Backend:

Feature	Product area	Priority	Complexity	Description, user/data flow, validations, UI surfaces, backend/data, tools, security, ops
				invoicing service, numbering service, posting service, email service. Tools: Tiptap for notes/terms, Resend https://resend.com/pricing for delivery, PostgreSQL RLS. Controls: maker-checker on send/post, immutable numbering, audit of edits before posting. Ops: PDF generation and reminder volume drive costs.
Purchase bills, debit notes, AP	Accounting	P0	M	AP clerk creates bill, allocates expense/asset accounts and tax treatment, reviews duplicates, then posts to AP/expense ledgers. Data: vendors, purchase_bills, purchase_bill_lines, debit_notes, attachments. Validate duplicate bill heuristics, vendor tax ID rules, open period, and account eligibility. UI: bills list, create bill, vendor detail, attachment preview. Backend: AP service, attachment pipeline, posting engine. Tools: Uppy, SeaweedFS or Cloudflare R2. Controls: supplier document audit, malware/file scanning is recommended, role-based posting rights. Ops: storage growth and OCR are major future cost drivers.
Payments, receipts, cash, allocations	Accounting	P0	M	User records or imports receipt/payment, allocates against invoices/bills, handles partials, write-offs, fees, and reversals. Data: payments, payment_lines, allocations, bank_accounts, cash_accounts. Validate allocation totals, counterparties, cash/bank account existence, lock periods, and reversal permissions. UI: payment list, payment wizard, allocation grid. Backend:

Feature	Product area	Priority	Complexity	Description, user/data flow, validations, UI surfaces, backend/data, tools, security, ops
				settlement service, allocation engine, posting engine. Tools: React Hook Form , Turn2search1 , TanStack Table , Turn2search0 . Controls: dual approval above thresholds, masked bank details, full audit trail. Ops: concurrency around partial allocations must be handled carefully.
Bank statements and reconciliation	Accounting	P1	H	User imports statement or bank feed, system suggests matches, user accepts/rejects/creates adjustments, then reconciled state updates bank ledger and outstanding documents. Data: bank_statements, statement_lines, reconciliation_runs, reconciliation_matches. Validate one-to-one/one-to-many constraints, unreconciled carryovers, duplicate feed imports, and open-period behavior. UI: high-density reconciliation workspace. Backend: import parser, matching engine, confidence scoring, adjustment posting. Tools: Tally bank reconciliation help , ERPNext reconciliation docs , Odoo bank reconciliation docs , LedgerSMB reconciliation reference . Controls: maker-checker for write-offs, statement immutability after lock. Ops: matching algorithms and statement volume drive compute/storage needs.
GST/VAT/tax configuration and filing	Accounting	P1	H	Tax manager configures codes/rates/registrations; invoices and bills compute tax entries; systems generate return workpapers and reconciliation reports. Data: tax_codes, tax_rates, tax_rules, tax_returns, tax_reconciliations. Validate effective dates, jurisdiction conflicts, exemption

Feature	Product area	Priority	Complexity	Description, user/data flow, validations, UI surfaces, backend/data, tools, security, ops
				logic, filing-period locks. UI: tax settings, returns list, reconciliation reports. Backend: tax engine, filing export service, localization service. Tools: Tally GSTR reconciliation help, Tally release notes, Odoo taxes docs, Odoo fiscal localizations. Controls: maker-checker, filed-period locks, export signing where required, audit logs. Ops: localization breadth is the biggest scope risk and is unspecified.
Reporting, audit, period close	Accounting	P0	M	Reports derive from posted journals and document balances: trial balance, GL, P&L, balance sheet, aging, tax workpapers. Period close checks unresolved items then locks the period. Data: materialized/reporting views over journals plus audit_events, period_closures. Validate close blockers, report filters, and restatement/reopen permissions. UI: report hub, report detail, close checklist, audit log. Backend: reporting query service, BI layer, export generation. Tools: Metabase OSS, PostHog for product analytics, Matomo if privacy-first web analytics are preferred. Controls: period lock, maker-checker, export audit, backup before close. Ops: large GL exports and heavy BI queries can exceed free tiers quickly.
Imports, migration, APIs, admin	Accounting	P1	M	CSV/ERP imports map legacy data into masters, opening balances, invoices, and journals; APIs and webhooks expose controlled integration points. Data: import_jobs, import_mappings, api_keys, webhook_endpoints, admin_audit. Validate column mappings, idempotency,

Feature	Product area	Priority	Complexity	Description, user/data flow, validations, UI surfaces, backend/data, tools, security, ops
				duplicate imports, and reference integrity. UI: imports center, key management, webhook logs, tenant admin. Backend: import workers, token service, audit service. Tools: Keycloak for enterprise IAM path, BullMQ when Redis workers are justified. Controls: scoped keys, webhook signatures, tenant isolation, retention policy. Ops: migration quality is a make-or-break go-live factor.
CRM and customer graph	Business OS AI	P0	M	Leads arrive from forms/imports/manual entry, convert to contacts/accounts/deals, and later feed proposals and invoices. Data: accounts, contacts, leads, deals, activities, tags. Validate dedupe, stage transitions, ownership, and required monetary fields. UI: contacts list/detail, deal pipeline, activity timeline. Backend: CRM service, event stream, dedupe jobs. Tools: TanStack Table, shadcn/ui. Controls: RLS, activity audit, role-based visibility. Ops: dedupe/search quality drives perceived platform intelligence.
Website/CMS, forms, publishing	Business OS AI	P0	H	Marketer creates site/page, edits sections, collects leads via forms, previews, publishes to domain/subdomain, and tracks submissions. Data: sites, pages, page_versions, forms, submissions, domains, deployments. Validate slug uniqueness, publish preflight, domain verification, and draft/live versioning. UI: site list, editor, SEO settings, publish center. Backend: CMS service, deploy/publish service, form ingestion, asset pipeline. Tools: Next.js, Tiptap,

Feature	Product area	Priority	Complexity	Description, user/data flow, validations, UI surfaces, backend/data, tools, security, ops
				?url?Uppy?turn10search0? , ?url?Cloudflare Pages docs?https://developers.cloudflare.com/pages/? . Controls: draft/live separation, audit trail for publish/rollback, CSP and secure uploads. Ops: image/file egress and build frequency are routine cost drivers.
Projects, tasks, files, portal	Business OS AI	P0	M	Teams plan work, attach files, comment, share client-safe views, and optionally expose selected artifacts in a client portal. Data: projects, tasks, comments, files, permissions, portal_sessions. Validate file permissions, due dates, external visibility, and task dependencies if used. UI: project workspace, task board/list, file manager, portal pages. Backend: project service, comment service, file metadata service. Tools: ?url?Uppy?turn10search0? , ?url?SeaweedFS?turn5search1? , ?url?CodeMirror?turn11search1? for structured notes/rules where needed. Controls: RLS, encrypted object storage, signed downloads, audit of client shares. Ops: storage, previews, and search indexing grow steadily.
Proposals, contracts, billing ops	Business OS AI	P1	M	Sales creates proposal, edits sections and pricing, routes for approval, sends to client, then converts accepted work into invoice/subscription records. Data: proposals, proposal_versions, approvals, billing_profiles, linked accounting docs. Validate pricing math, approval thresholds, version integrity, and conversion idempotency. UI: proposal editor, approvals, send/review, invoice conversion. Backend: document service, approval service, bridge into accounting AR. Tools: ?url?Tiptap?turn10search1? or ?url?Lexical?turn10search2? ; email via

Feature	Product area	Priority	Complexity	Description, user/data flow, validations, UI surfaces, backend/data, tools, security, ops
Automations, integrations, analytics	Business OS AI	P1	M	Resend. Controls: maker-checker on approvals, version history, legal/audit retention. Ops: PDF generation and signature integration later are complexity multipliers. Event-driven workflows trigger emails, assignments, reminders, updates, and webhooks; product analytics and BI expose funnel and usage trends. Data: workflow_defs, workflow_runs, events, webhooks, dashboards, saved_reports. Validate idempotency, retry policy, event schema, user scopes. UI: automation builder, run logs, dashboard/report pages. Backend: event bus/outbox, job workers, analytics pipeline. Tools: pg-boss, BullMQ, PostHog, Metabase. Controls: signed webhooks, replay protection, audit of automation side effects. Ops: analytics event volume can outgrow free tiers early.
AI copilot and knowledge retrieval	Business OS AI	P1	H	User prompts the copilot with page context or selected records; system retrieves tenant-scoped docs/data, drafts emails/proposals/summaries, and asks for confirmation before any side effect. Data: knowledge_docs, chunks, ai_threads, ai_runs, feedback, tool_calls. Validate tenant scope, allowed tools, prompt budgets, and human approval for customer-facing or financial actions. UI: docked AI panel, context picker, cited answer cards, confirmation drawer. Backend: RAG service, model gateway, rate limiting, cost logging. Tools: LiteLLM, vector store choice is unspecified, text editors via Tiptap. Controls:

Feature	Product area	Priority	Complexity	Description, user/data flow, validations, UI surfaces, backend/data, tools, security, ops
				RLS, prompt-injection defenses, tool allowlists, audit/evals per NIST AI RMF and OWASP LLMSSVS . Ops: model cost and retrieval latency are the key levers.
Optional media and live events	Business OS AI	P3	H	If later needed, users upload media, system performs resumable ingest, transcodes/packages streams, then serves playback or live sessions. Data: media_assets, renditions, playback_tokens, streams, recordings. Validate file types, duration/size, entitlement, and playback token expiry. UI: asset library, upload queue, player/embed settings. Backend: ingest, transcode, packaging, playback authorization. Tools: tusd , FFmpeg , Shaka Packager , Video.js , OvenMediaEngine . Controls: signed URLs/tokens, storage isolation, abuse throttling. Ops: storage reads and video delivery are where free tiers fail first.

Accounting data flow

flowchart LR

```

A[Sales invoice / purchase bill / payment / journal entry]
B[Validation layer<br/>party, tax, period, debit-credit, numbering]
C[Approval / maker-checker]
D[Posting engine]
E[(Journal entries + journal lines)]
F[(General ledger / subledgers)]
G[(AR/AP balances)]
H[(Tax workpapers / returns)]
I[(Reports: TB, GL, P&L, Balance Sheet, Aging)]
J[Bank statements]
K[Reconciliation engine]

```

```

A --> B --> C --> D --> E --> F --> I
E --> G --> I

```

E --> H --> I
J --> K --> D
G --> K

Overall platform feature map

mindmap

```
root((Combined Platform))
  Accounting Core
    Tenant and Fiscal Setup
    Chart of Accounts
    Journals and Vouchers
    AR Invoices and Credit Notes
    AP Bills and Debit Notes
    Payments and Allocations
    Bank Reconciliation
    GST/VAT and Returns
    Reports and Period Close
    Imports APIs Admin
  BusinessOS AI
    CRM and Deals
    Website CMS Publishing
    Forms and Lead Capture
    Projects Tasks Files
    Proposals and Contracts
    Client Portal
    Automations and Integrations
    Analytics and Dashboards
    AI Copilot and Knowledge
    Optional Media
```

Implementation roadmap

Phase	Milestone	Deliverables	Person-weeks
MVP 30–60 days	Accounting foundation	tenant/company setup, COA, journals/vouchers, invoice/bill creation, payments, basic AR/AP, trial balance/GL/P&L, audit log, period open/close v1	16–20
MVP 30–60 days	BusinessOS engagement layer	auth/workspaces, CRM core, website/CMS/forms, project/tasks/files, invoice/proposal bridge to accounting, basic analytics	14–18
V1	Control and reconciliation	bank import/reconciliation, GST/VAT config and returns for one jurisdiction, maker-checker approvals, richer reports, webhook/API layer, portal polish	12–16

Phase	Milestone	Deliverables	Person-weeks
V1	AI and automations	tenant-scoped AI copilot, workflow builder, event bus/outbox, usage metering, admin console, BI and saved dashboards	10–14
V2	Localization and scale	extra country packs, enterprise IAM, advanced billing/subscriptions, search and document intelligence, reconciliation heuristics, performance hardening	14–20
V2	Optional media and advanced integrations	resumable uploads, asset processing, playback/live, partner integrations, marketplace/dev portal expansion	8–14

Total indicative effort: MVP 30–38 person-weeks, V1 22–30 person-weeks, V2 22–34 person-weeks. Because team size and budget are **unspecified**, these are coordination-neutral estimates rather than elapsed-time promises.

Security, compliance, and operations summary

The accounting core should default to strict tenant isolation with [PostgreSQL RLS](#) on pooled tables and dedicated schemas or databases for premium tenants when needed. Every financial mutation should be auditable, reviewable, and reversible through approved workflows rather than direct edits. Maker-checker should be required for posting, filing, closing periods, and high-value disbursements. Period locks, immutable journal history, attachment retention, encrypted transport/storage, scoped API keys, and signed webhooks are baseline requirements. The technical control framework should be checked against [OWASP ASVS](#); the AI layer should follow [OWASP LLMSVS](#) and [NIST AI RMF](#); and UI accessibility should follow [WCAG 2.2](#) and [WAI-ARIA APG](#). Operationally, the first cost drivers are not CPU alone but analytics event ingestion, file/object storage and reads, PDF/export jobs, statement ingestion, and eventually AI tokens and media egress where enabled. [\[4\]](#)

Final recommendation

If the platform must launch in phases, start with **Accounting Core + CRM + invoices/proposals bridge + website/forms**. That gives you the strongest business spine: revenue capture, posting, customer records, and lead generation. Add reconciliation, tax, automations, analytics, and AI next. Leave optional media and broad localization until after one jurisdiction and one accounting operating model are stable. TallyPrime’s own help, together with ERPNext, Odoo, Akaunting, and LedgerSMB, all show that the accounting moat is created by posting correctness, reconciliation, tax support, and reports—not by breadth alone. BusinessOS AI becomes materially better when it is grounded in that accounting truth rather than floating above it. [\[5\]](#)

[1] [3] [5] How to Record Accounting Entry in TallyPrime | TallyHelp

https://help.tallysolutions.com/accounting-entry-tally/?utm_source=chatgpt.com

[2] Next.js Docs: App Router

https://nextjs.org/docs/app?utm_source=chatgpt.com

[4] PostgreSQL: Documentation: 18: 5.9. Row Security Policies

https://www.postgresql.org/docs/current/ddl-rowsecurity.html?utm_source=chatgpt.com